

Divide and conquer

Problem flavors

- Binary search on array
- Binary parametric search
- Divide, process each, then merge

Binary search on array

FindFirstB

- Find the index of the first B given the array is of the form AA BBB . . .
- Or return -1 if B doesn't exist.

Examples

Input	Output
AABBBB	2
AAAA	-1
BBB	0

Reduction example

- **Binary search.** Find target in sorted array, or return -1 if not found.
- **Find peak element.** See homework!

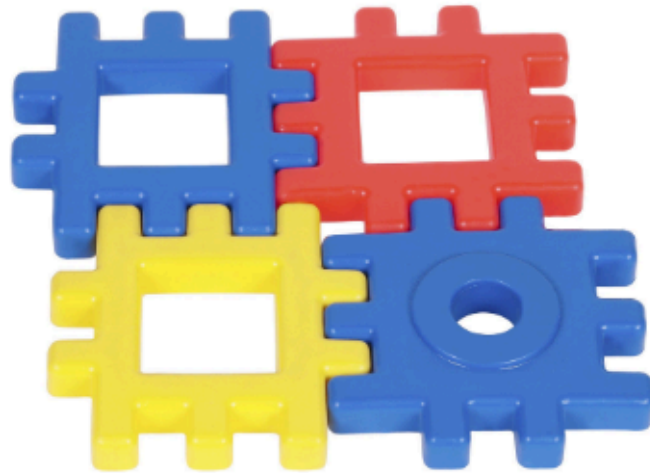
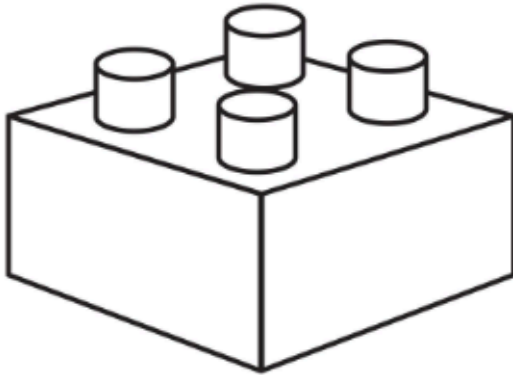
Implementing FindFirstB: Function specification

`findFirstB(char[] arr, int l, int r)`

- **Def 1: End inclusive.** Find the first 'B' in `arr[l..r]`, else return -1
- **Def 2: End exclusive.** Find the first 'B' in `arr[l..r-1]`, else return r

```
arr = "AABBB"  
findFirstBEndInclusive(arr, 0, 2) = "Search inside AAB"  
findFirstBEndExclusive(arr, 0, 2) = "Search inside AA"
```

Why be so specific?



Def 1: End inclusive

```
# Function specification: TODO(1)
findFirstB(arr, l, r):
    # Base case. TODO(4)

    mid = (l + r) // 2
    if arr[mid] == 'B':
        # TODO(2)
    else:
        # TODO(3)
```

Def 1: End inclusive

```
# find first B in arr[l..r] else -1
findFirstB(arr, l, r):
    # Base case
    if l > r: return -1
    if l == r: return l if arr[l] == 'B' else -1

    mid = (l + r) // 2
    if arr[mid] == 'B':
        return findFirstB(arr, l, mid)
    else:
        return findFirstB(arr, mid + 1, r)
```

Def 2: End exclusive

```
findFirstB("AA", 0, 2) = 2 # If not found return r
findFirstB("AB", 0, 2) = 1

# Function specification: TODO(1)
findFirstB(arr, l, r):
    # Base case. TODO(4)

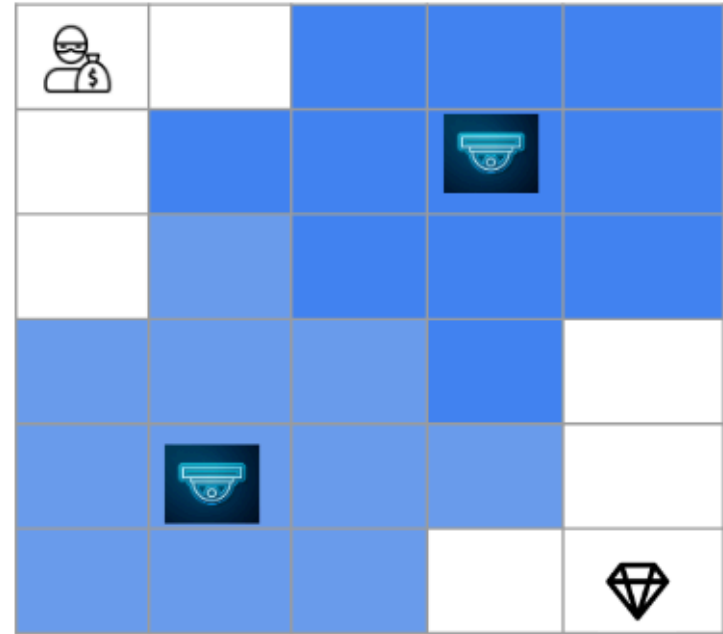
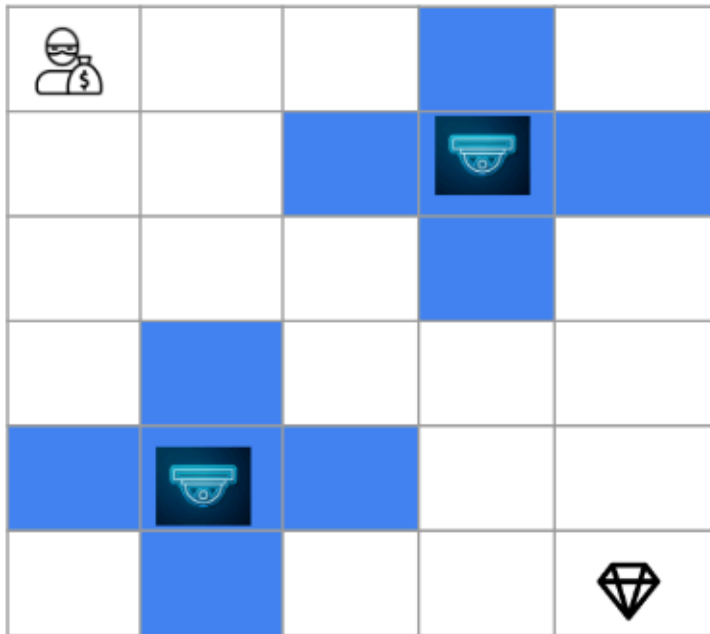
    mid = (l + r) // 2
    if arr[mid] == 'B':
        # TODO(2)
    else:
        # TODO(3)
```

Def 2: End exclusive

```
# find first B in arr[l..r-1], else r
findFirstB(arr, l, r):
    # Base case
    if l == r: return r

    mid = (l + r) // 2
    if arr[mid] == 'B':
        return findFirstB(arr, l, mid)
    else:
        return findFirstB(arr, mid + 1, r)
```

Parametric search: Stop the thief



What to binary search on?

Parameter (sensor power)	Is thief stopped?
0	No
1	No
2	Yes
3	Yes
...	...

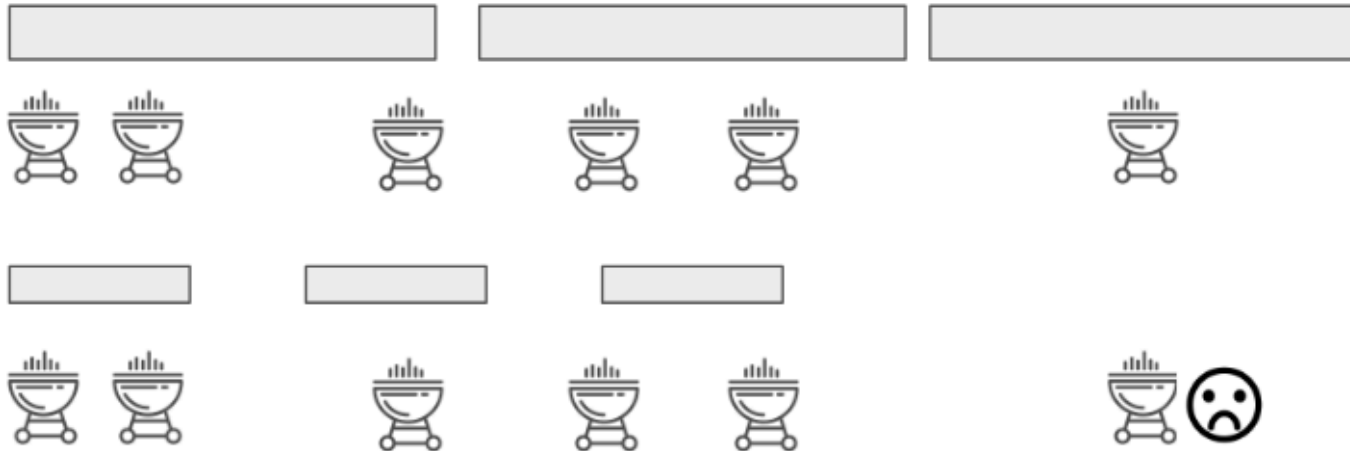
Parametric search

- Define `bool isFeasible(parameter)`
- Check: IsFeasible monotone?
- Binary search `parameter`

Pseudocode

```
findFirstB(  
    arr[i] = isThiefStopped(sensor coverage = i)  
    l = 0  
    r = grid size)
```

Umbrellas for bbq



Pseudocode

```
findFirstB(  
    arr[i] = isEveryoneCovered(umbrella size = i)  
    l = 0  
    r = field_length / num_umbrella)
```

Divide, process each, merge

Merge sort

https://en.wikipedia.org/wiki/Merge_sort

<https://opensa-server.cs.vt.edu/embed/mergesortAV>

Runtime analysis

Questionnaire

- Recursion tree height
- Total work at each level
- Total work = sum of total work at each level

Binary search runtime analysis

Spot the thief runtime analysis

Assume: `isThiefStopped(sensorPower)` is $O(N^2 S)$

Merge sort runtime analysis

Master theorem

$$T(n) = aT(n/b) + f(n)$$

[Wikipedia](#)

IRL

- Loops > recursion, if single threaded
- Parallelization. Divide and conquer v good.